

Pycon 2026 / May 2026

Pydantic Monty: Wild LLMs, from tool calling to computer

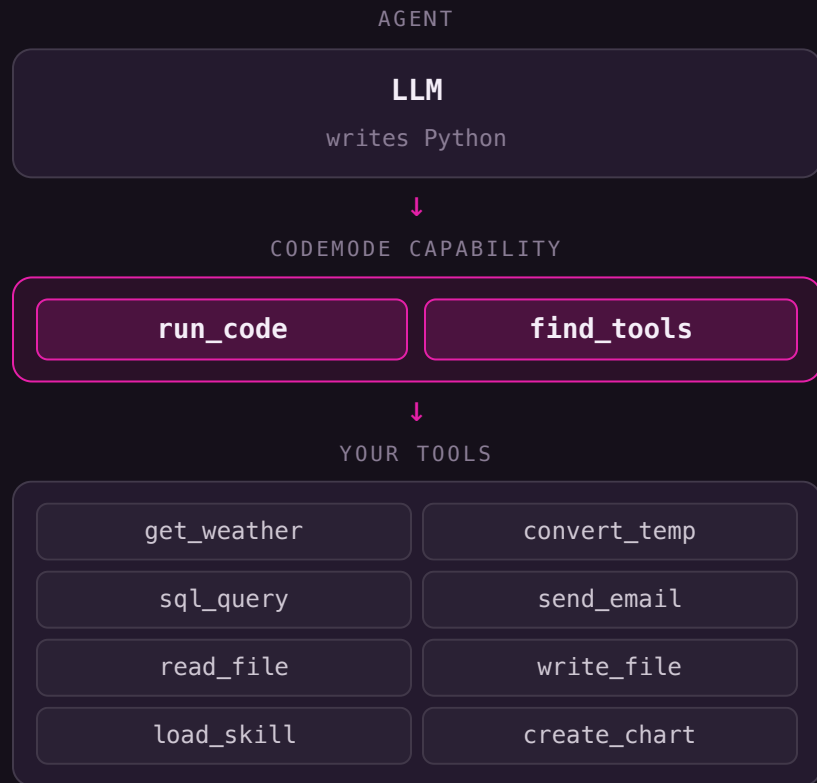
Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic**
- Founder & CEO of **Pydantic Labs**
- Working on **Pydantic AI**, **Logfire**, and **Monty**
- Writing Python and Rust developer
- Now AI talking head (like everyone else)

What is "codemode"?

- aka "Programatic tool calling", CodeAct, and RLM (Recursive Language Models)
- The model writes Python (or js); the Python calls tools
- Tool composition, branching, looping - all happen in **code**, not round-trips
- Intermediate state stays in the runtime, not in the context window
- The agent only ever sees **two** tools: `run_code` and `find_tools`
- "1000 tools in 1000 tokens" is the promise



But how do we run it?

THERE ARE MULTIPLE OPTIONS



The tradeoff

- A **curated list** beats the kitchen sink
- Most agents don't need a whole Linux box - they need ~5 tools used well
- "Analyse BI data to find efficiencies" sounds broad - the actual surface is tiny
- Full sandboxes solve a problem most embedded agents don't have

The sandbox vs. the desert

THE SANDBOX

sql_query

create_chart

create_table

load_skill

THE DESERT

awk sed gcc g++ clang less more
grep egrep fgrep make cmake
bash zsh sh fish curl wget
tar gzip bzip2 xz zip unzip
find xargs locate vim nano
emacs ed python3 pip pipx node
npm pnpm yarn git svn hg ssh
scp rsync cron systemd apt
apt-get dpkg yum dnf brew jq yq
tr cut sort uniq head tail

Introducing Monty

- A **minimal Python interpreter, written in Rust**, for code written by AI
- Safe by default: no filesystem, env, or network unless you grant it
- Pass in your own functions - the model calls them as ordinary Python
- **Type-checks** code (via `ty`) before it runs
- **Snapshottable** - pause at an external call, resume later, anywhere
- Callable from **Rust, Python, or JavaScript**

Tradeoff: Monty will not run your existing Python app - it's a Python *subset*.

```
uv add pydantic-monty / npm i @pydantic/monty
```


The advantages

STARTUP LATENCY

5 μ s
MONTY

VS.

~~1s~~
FULL SANDBOX

SNAPSHOT SIZE

1KB
MONTY

VS.

~~1GB~~
FULL SANDBOX

COST PER RUN

\$0
MONTY

VS.

~~\$?~~
FULL SANDBOX

Mini Demo



How Monty works

- Five phases
- First 3 phases can be reused
- The pause/resume protocol is what makes Monty unusual

1 Parse ruff → AST



2 Resolve names strings → integer slot ids



3 Compile AST → bytecode



4 Run heap + stack VM



5 Pause / Resume snapshot ↔ host

Step 1 · Parse to AST

- **ruff** parses the source — same AST as Python tooling
- Monty walks the AST eagerly into its own `ParseNode` / `Expr`
- Exit early if we encounter a syntax error, or syntax we don't yet support

```
greeting = 'Hello'  
who = get_who(greeting)  
f'{greeting} {who}'
```

```
Module {  
  body: [  
    Assign { target: Name("greeting"), value: Constant("Hello") },  
    Assign {  
      target: Name("who"),  
      value: Call {  
        func: Name("get_who"),  
        args: [Name("greeting")]  
      }  
    },  
    Expr(FString {  
      parts: [Name("greeting"), " ", Name("who")]  
    })  
  ]  
}
```

Step 2 · Resolve names

- Second pass classifies every name: **global**, **local**, **free** (closure), or **LocalUnassigned**
- String keys become integer **NamespaceId** slot indices
- **LocalUnassigned** = name read but never bound → host-provided external function
- Closure-cell capture decided here so nested functions can reach enclosing locals

NAME	CLASSIFICATION	SLOT
'Hello'	const	consts[0]
get_who	LocalUnassigned	– (host)
greeting	local	locals[0]
who	local	locals[1]

Step 3 • Compile to bytecode

- Convert the AST into **bytecode** instructions
- Effectively a `list[int]` - where each `int` is an opcode and its operands
- This compile step is similar to cpython - why we have `.pyc` files

```
greeting = 'Hello'
who = get_who(greeting)
f'{greeting} {who}'
```

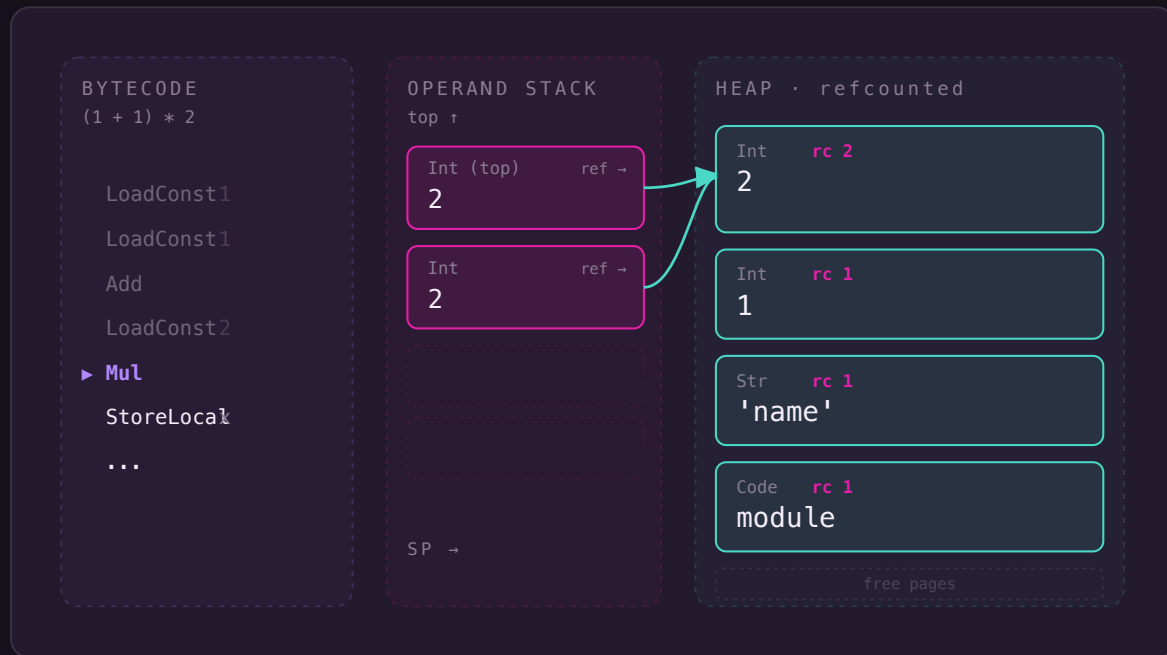
IP	OP	ARG
0	LoadConst	'Hello'
1	StoreLocal	greeting
2	LoadName	get_who host
3	LoadLocal	greeting
4	Call	1
5	StoreLocal	who
6	LoadLocal	greeting
7	LoadLocal	who
8	FormatStr	"{} {}"
9	Return	None

Step 4 • Run on a heap + stack VM

- Execute the bytecode - iterate over the `list[int]` "doing each thing"
- much like other interpreters
- Two stacks: an **operand stack** for values and a **CallFrame stack** for activations

RUNNING

```
x = (1 + 1) * 2
```



Step 5 · Pause for the host, resume, repeat

- Run the code until we hit an external call
- Then return to the host
- Ask it to call the function

```
class Monty:
    def start(self) -> CallTool | Done: ...
    def resume(self, result: ReturnValue | Exception) -> RunProgress: ...

run = Monty(code)
progress = run.start()

while isinstance(progress, CallTool):
    # e.g. CallTool("get_who", ("Hello",))
    # we could call run.dump() here if we wanted to
    result = call_tool(progress.name, progress.args)
    progress = run.resume(result)

print(progress)
```


Hack Monty

- Public bounty at hackmonty.com
- Scape the sandbox, read the secret and we'll pay **\$5,000**
- Launched 3 weeks ago
- Vulnerability found via use-after-free bug, and lots of less severe issues
- Relaunching in a week with double the prize - **\$10,000**

Hack Monty

Welcome. This is a honeypot. The server behind it executes whatever Python you POST to /run/ inside [pydantic monty](#), our language-level sandbox.

There is a secret on this machine. Your job is to find it — by escaping the sandbox. If you do, we'll pay you \$5,000 USD.

[Blog post about this.](#)

[Also look at the API docs.](#) (or [Redox](#))

[Grab the hackmonty.py CLI](#) to run code here from your terminal.

[View the traces](#) in Pydantic Logfire to see how your code is executed (and everyone else's!).



Demo



Thank you

Find more at

github.com/pydantic/talks/tree/main/2026-05-pycon



@samuelcolvin · @pydantic